

Prompt Engineering Take-Home Assignment

Context

As AI capabilities advance rapidly, engineers at Anthropic face a unique challenge: building tools and experiences that are genuinely useful, trustworthy, and delightful. The best products demonstrate taste, handle complexity gracefully, and solve real problems in non-obvious ways.

This assignment asks you to demonstrate your prompt engineering and evaluation skills through a focused, practical task.

Time Expectation

Target: 1-2 hours. Hard limit: 8 hours.

This assignment is designed to replace a synchronous interview session. We expect most candidates to complete something compelling in 1-2 hours, and we value your time — something insightful and deeper is more valuable than breadth. You may spend up to 8 hours if you're enjoying the problem, but this is not expected or required—we've seen excellent submissions completed in under 2 hours.

If you find yourself exceeding 8 hours, stop and submit what you have. Part of what we're evaluating is your ability to scope effectively.

In your written rationale, please note approximately how long you spent.

The Task

Build a system that uses Claude and Wikipedia to answer questions, and evaluate how well it works.

Specifically, you will:

1. **Design a system prompt and tool definition** for an LLM that has access to a `search_wikipedia(query: str)` tool. Your prompt should guide the

model to use Wikipedia effectively to produce high-quality answers to user questions.

2. **Wire up a basic Wikipedia retrieval integration** so the system actually works end-to-end. You may use any approach: the [MediaWiki API](#), a downloaded Wikipedia dump, a local search index, or any other method.
3. **Build an eval suite** that measures how well your system works across a range of test cases.

Requirements

Deliverables

Create three deliverables:

1. A runnable prototype

- The Anthropic team should be able to run your system and interact with it
- A CLI, notebook, or script that takes a question and returns an answer (showing whether search was used) is sufficient
- Must include clear setup instructions (dependencies, API keys needed, etc.)
- Include sample queries or a demo mode so the reviewer can see it work immediately

2. Your code

- Submit via GitHub repo link
- Code quality matters but is not the primary evaluation criterion - Use any languages/technologies you're comfortable with

3. Design rationale

- Use both formats: self-recorded video (~5 min) and a short written doc
- Your prompt engineering approach and why you made the choices you did
- How you designed your eval suite: what dimensions of quality you chose to measure, and why
- Where your system succeeds and where it fails — what did you learn from the evals?
- Key iterations you made based on eval results
- How you'd extend this with more time
- Approximately how long you spent

Constraints

- You must use an **Anthropic model** via the Anthropic API. Please specify which model you used (e.g., Claude Sonnet 4, Claude Haiku 3.5) in your writeup.
- You **must not** use built-in hosted search or RAG tools (e.g., Anthropic's `web_search` tool type, OpenAI's web browsing, Perplexity, etc.) — we want to see how you design the system yourself
- The Wikipedia data source is your choice (live API, dump, local index, etc.) - Focus on **prompt quality** and **eval design**, not on building a production search system

Use of AI During Development

You are expected to use Claude Code, Claude.ai, or other AI tools during development. We want to see how you leverage these tools effectively — this is a core skill for engineers at Anthropic.

Please submit your Claude/AI transcripts alongside your code. We're evaluating your judgment: how you direct the AI, evaluate its outputs, make tradeoffs, and maintain your own vision. A fully AI-generated project with no judgment will not pass.

Submission

Please submit via the GreenHouse link:

- GitHub repo link with your code
- Clear instructions for running the prototype
- Claude/AI transcripts showing your interaction history during development.
You can generate these using a tool like [Claude Code transcripts](#) or the "Share" button on Claude.ai.
- Explanation artifacts (video + short written doc)